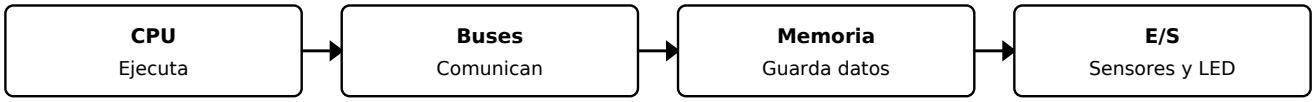
Resumen de la semana 1

Del computador al microcontrolador · Bits y E/S mapeada en memoria

1. Del computador al microcontrolador

Un **sistema embebido** es un equipo que incorpora un controlador para cumplir una función concreta, por ejemplo, una alarma, una lavadora o un termostato. Normalmente tiene pocos recursos y necesita responder a las entradas que recibe.

Un **microprocesador** se centra en la CPU y suele necesitar memorias y dispositivos externos. Un **microcontrolador**, en cambio, reúne en un mismo chip la CPU, la memoria y los periféricos; por eso es práctico para sistemas pequeños y de bajo consumo.



La CPU contiene una unidad aritmético-lógica (ALU), registros rápidos, una unidad de control y el contador de programa (PC). Trabaja en un ciclo que se repite: **buscar una instrucción, decodificarla y ejecutarla**. En el ATmega328P del Arduino Uno, el reloj es de 16 MHz.

Bus	Para qué sirve	Ejemplo
Direcciones	Indica qué posición se quiere usar.	0x25
Datos	Transporta el valor que se lee o se escribe.	0010 0000
Control	Indica la operación.	Leer / escribir

Arquitecturas y memorias

En **von Neumann** el programa y los datos comparten el camino hacia la CPU; en **Harvard** hay memorias o buses separados y puede accederse a ambos de forma paralela. El ATmega328P usa arquitectura Harvard.

Memoria	Qué guarda	¿Se pierde al apagar?
Flash	El programa que se carga en la placa.	No
SRAM	Variables y datos mientras se ejecuta.	Sí
EEPROM	Valores pequeños que deben permanecer.	No

Flash y EEPROM soportan un número limitado de escrituras. Por eso no conviene grabar continuamente en EEPROM dentro de `loop()`.

ATmega328P y ESP32

Característica	ATmega328P / Arduino Uno	ESP32 / DevKit V1
Procesamiento	8 bits; un núcleo a 16 MHz	32 bits; dos núcleos, hasta 240 MHz
Memoria	32 KB Flash; 2 KB SRAM	4 MB Flash; aprox. 520 KB SRAM
Niveles lógicos	5 V	3,3 V; no tolera 5 V
Conectividad	Sin Wi-Fi ni Bluetooth integrados	Wi-Fi y Bluetooth

2. Bits, números y operaciones lógicas

Un **bit** representa 0 o 1. Ocho bits forman un **byte**, con pesos 128, 64, 32, 16, 8, 4, 2 y 1. Para pasar de binario a decimal se suman únicamente los pesos donde aparece un 1.

El sistema **hexadecimal** usa los dígitos 0-9 y A-F. Cada símbolo hexadecimal representa cuatro bits. Se escribe con prefijo `0x`: por ejemplo, `0010 0000` = `0x20` = 32.

Operación	En C	Qué ocurre
AND	`&`	1 solamente si ambos bits son 1.
OR	`|`	1 si por lo menos uno es 1.
XOR	`^`	1 cuando los bits son diferentes.
NOT	`~`	Invierte los bits.

Para cambiar un solo bit se usa la **máscara** `1 << n`, que coloca un 1 en la posición `n`. La ventaja es que se pueden modificar pines sin alterar los demás bits del registro.

Acción	Instrucción	Idea
Encender bit n	`REG |= (1 << n);`	OR pone ese bit en 1.
Apagar bit n	`REG &= ~(1 << n);`	AND con la máscara invertida lo pone en 0.
Invertir bit n	`REG ^= (1 << n);`	XOR cambia 0 ↔ 1.
Consultar bit n	`REG & (1 << n)`	Da distinto de cero si está activo.

3. Control de pines en AVR

Los registros de **entrada y salida mapeados en memoria** tienen una dirección; al escribir sus bits se modifica el comportamiento físico de los pines. En AVR se usan tres registros por puerto:

Registro	Función	Bit en 1
DDRx	Selecciona dirección de cada pin.	Salida
PORTx	Escribe la salida o activa pull-up si es entrada.	Alto / pull-up
PINx	Lee el nivel presente en el pin.	Nivel alto

En un pin configurado como entrada con **pull-up**, el botón conectado a GND se lee en `HIGH` cuando está suelto y en `LOW` al presionarlo. No hay que confundir lo leído en `PINx` con lo escrito en `PORTx`.

```
// LED del Arduino Uno: pin 13 = PB5
DDRB  |=  (1 << 5);       // PB5 como salida
PORTB |=  (1 << 5);       // encender LED
PORTB &= ~(1 << 5);       // apagar LED

// Boton en PD2, con resistencia pull-up interna
DDRD  &= ~(1 << 2);       // entrada
PORTD |=  (1 << 2);       // activa pull-up
bool pulsado = !(PIND & (1 << 2));
```

4. Ejercicios y ejemplos resueltos

Conversión 1. `1000 0001` = 128 + 1 = **129 decimal**; en hexadecimal, `0x81`.

Conversión 2. `0xFF` = `1111 1111` = **255 decimal**.

Conversión 3. `200` = 128 + 64 + 8 = `1100 1000` = `0xC8`.

Conversión 4. `0x08` = `0000 1000`; el bit que está en 1 es el **bit 3** (se cuenta desde 0).

Para los siguientes ejercicios, cada operación parte de `REG = 1010 1010` (0xAA), tal como aparece en la actividad de clase:

Operación	Resultado	Explicación
`REG |= (1 << 0)`	`1010 1011`	Se enciende el bit 0.
`REG &= ~(1 << 7)`	`0010 1010`	Se apaga el bit 7.
`REG ^= 0x0F`	`1010 0101`	Se invierten los cuatro bits bajos.
`REG & (1 << 3)`	Verdadero	El bit 3 ya vale 1.
Encender bits 0 y 1	`REG |= 0x03;`	Resultado: `1010 1011`.

¿Por qué no `REG = (1 << 2)`**?** Porque **reemplaza todo el registro** por `0000 0100`; no conserva los demás bits. Para encender únicamente el bit 2 se usa `REG |= (1 << 2);`.

Predicción del pin. Con `DDRB5 = 1` y `PORTB5 = 1`, PB5 es salida alta (5 V). Si `PORTB5 = 0`, la salida es baja (0 V). Con `DDRD2 = 0`, `PORTD2 = 1` y botón presionado a GND, `PIND2 = 0`.

Para repasar: distinguir los tres buses, decidir entre Flash/SRAM/EEPROM, convertir números de 8 bits y recordar la función de DDR, PORT y PIN. Son las bases para trabajar después con el ESP32.